

linalg: A Dense Numerical Linear Algebra Library in C++17

Aren Vista, Micah Havens, Sina Roomi, Max Monson, Jingson Guo, Dominic Mello

August 1, 2026

This project is a C++ library for numerical linear algebra. It provides a set of classes for performing operations on dense matrices and vectors — arithmetic, norms, factorization, linear solving, least squares, and eigenvalue computation — implemented from first principles rather than by wrapping BLAS/LAPACK. The library is designed to be efficient and easy to use, with a focus on exploring the techniques commonly taught in numerical linear algebra, and to serve as a demonstration of ability to design and build robust numerical software: choosing algorithms for stability rather than convenience, handling floating-point edge cases (overflow-safe norms, cancellation-free reflectors, deflation criteria), and verifying every component against the mathematical identities it must satisfy.

Complementary to the library is a set of notes that explain the algorithms and the rationale behind their computational methods. These notes can be accessed here: [Numerical Linear Algebra — L01: Basic Matrix Algebra](#).

What the Library Covers

- **Core types.** Owning `Matrix` / `Vector` classes over a scalar abstraction (`NumericTraits`) that supports `float`, `double`, and their complex counterparts uniformly, plus non-owning stride-aware views (`MatrixView`) so blocked algorithms can operate on panels without copying.
- **Computational kernels.** BLAS-shaped level 1/2/3 routines (`gemm`, `gemv`, `trsm`, ...) that serve as the single tuning surface for performance work, and the two workhorse orthogonal primitives — Householder reflectors and Givens rotations — stored implicitly and applied in $O(mn)$.
- **Direct factorizations.** LU with partial and full pivoting, Cholesky and LDL^H , Householder QR with and without column pivoting, and the condensed forms (Hessenberg, tridiagonal, bidiagonal) that feed the eigenvalue and singular value algorithms.
- **Eigenvalues and the SVD.** Symmetric eigensolving via implicit-shift QR with Wilkinson shifts, the real Schur form by Francis double-shift sweeps, general eigenvectors on top of it, and the Golub–Kahan SVD.
- **High-level solvers.** A dispatching `LinearSolver` that inspects the matrix and chooses a factorization, least squares by QR/SVD (with the normal equations included only as a cautionary comparison), and Gauss–Newton / Levenberg–Marquardt for nonlinear problems.

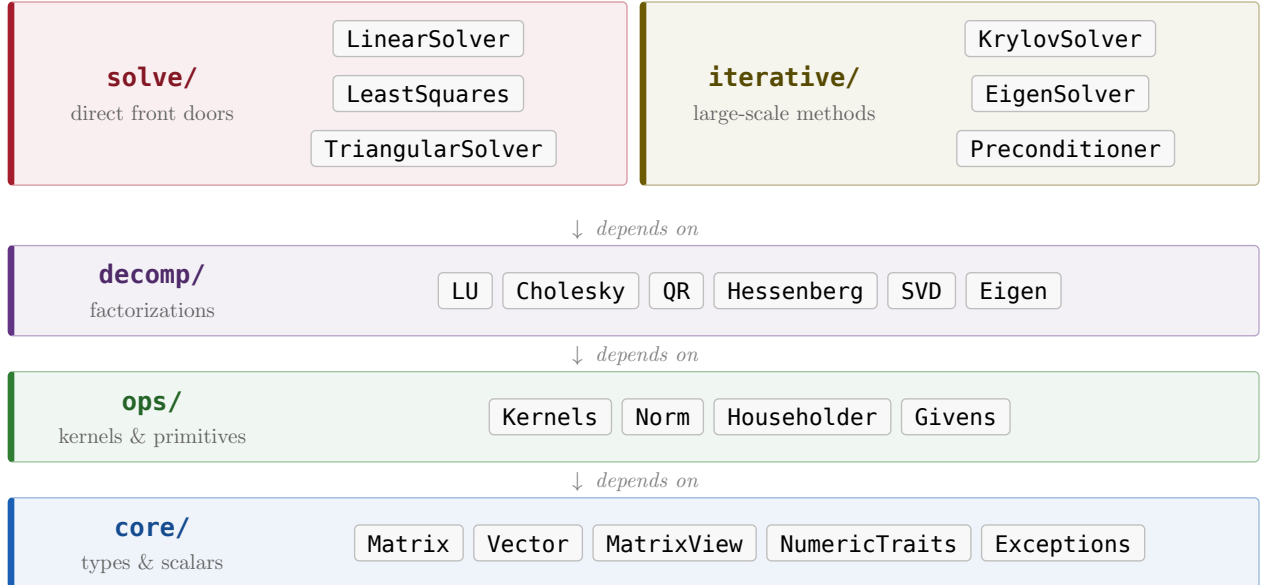


Figure 1: The dependency tiers of the library. Each tier is written only in terms of the tiers below it, so tuning the `ops` kernels accelerates every factorization and solver above without touching their code.

The Kernels as the Tuning Surface

“BLAS-shaped” means the kernels follow the interface conventions of the Basic Linear Algebra Subprograms, the de facto standard vocabulary of dense computation since the 1970s: strided vector arguments, an `alpha` / `beta` scaling pair, and in-place updates like $C \leftarrow \alpha AB + \beta C$ (`gemm`) rather than functions that allocate and return results. The *levels* classify routines by the ratio of arithmetic to data touched:

- **Level 1** — vector–vector, $O(n)$ work on $O(n)$ data: `axpy`, `dot`, `nrm2`. Memory-bound; there is little to tune.
- **Level 2** — matrix–vector, $O(n^2)$ work on $O(n^2)$ data: `gemv`, `ger`, `trsv`. Still memory-bound: every matrix element is read once and used once.
- **Level 3** — matrix–matrix, $O(n^3)$ work on $O(n^2)$ data: `gemm`, `syrk`, `trsm`. The only level with real data reuse — each element participates in $\sim n$ operations — which is what cache blocking, packing, and SIMD micro-kernels can exploit to approach the hardware’s peak floating-point throughput.

This is why the kernels are the *single tuning surface*: the factorizations above are written as thin orchestration around kernel calls (a blocked LU is a panel factorization plus `trsm` and `gemm` updates on the trailing submatrix). Optimizing `gemm` alone therefore accelerates LU, Cholesky, QR, and the condensed-form reductions at once, with no change to their code — mirroring how LAPACK gets its speed from whatever tuned BLAS it is linked against. It also bounds the risk of performance work: the naive triple-loop kernels stay as the reference implementation, and a tuned variant must merely reproduce their results to be correct everywhere.

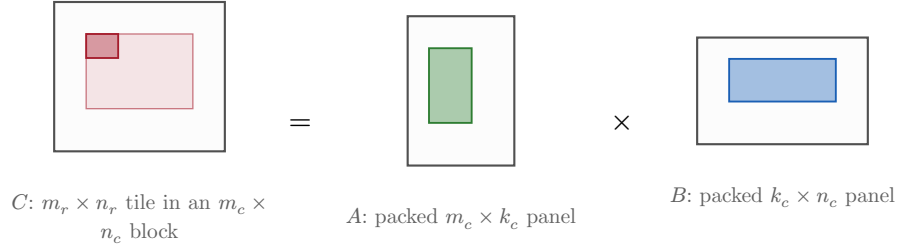


Figure 2: Level-3 blocking: `gemm` packs a panel of A and a panel of B into contiguous buffers sized for the cache hierarchy, then a register-tiled micro-kernel sweeps small tiles of C , reusing every packed element $\sim n$ times.

Design

Headers declare, source files define. Every class template is explicitly instantiated in `src/` for the supported scalar list, so the project builds as an ordinary compiled library with real separate compilation instead of a header-only template dump. Unimplemented members throw a `LinalgError` naming themselves, which makes the remaining work greppable and turns the test suite into a red-to-green progress meter.

Remark (Progress at a glance) . Every stub names itself when it throws, so the remaining work is one grep away — the per-file count doubles as a burndown chart:

```
grep -rc "not implemented" src/ | sort -t: -k2 -rn
```

Correctness is checked against identities rather than golden files: $PA = LU$, $Q^H Q = I$, residual and backward-error bounds sized by machine epsilon and the condition number, ill-conditioned stress inputs like the Hilbert matrix.

Directions for Expansion

Expanding this library to include methods of large-scale and structured computation is the natural next step, in rough order of ambition:

- **Iterative Krylov methods** (already scaffolded): conjugate gradients, GMRES with restarting, BiCGSTAB, and LSQR, with Jacobi, SSOR, and incomplete-Cholesky preconditioners — the bridge from dense $O(n^3)$ factorizations to problems where only matrix–vector products are affordable.
- **Iterative eigensolvers**: power iteration, Lanczos with selective reorthogonalization, and implicitly restarted Arnoldi for a few eigenpairs of large matrices.
- **Performance engineering**: cache-blocked and packed `gemm` with a register-tiled micro-kernel, blocked variants of LU/Cholesky/QR, and compact-WY application of reflector sequences, benchmarked against the naive implementations the correctness tests already validate.
- **Sparse storage and operations**, which the iterative methods are deliberately shaped to exploit (they touch the matrix only through products and preconditioner applications).

Building

```
cmake -B build -DLINALG_BUILD_TESTS=ON  
cmake --build build  
ctest --test-dir build
```